

Git workflows

COMPLETE LEARNING GUIDE

What You'll Learn

Git is the shared source of truth for how a team's code changes over time, and how well a team uses it — commit hygiene, branching strategy, merge discipline — has a direct, daily impact on how easy the codebase is to work in and debug. This guide covers writing commits that actually communicate intent, comparing the major branching strategies (Git Flow, trunk-based development, GitHub Flow), the practical difference between merging and rebasing and when each is appropriate, a calm process for resolving merge conflicts instead of panicking, safely undoing mistakes without losing work, and the repo hygiene practices (a proper `.gitignore`, never committing secrets) that prevent entirely avoidable incidents.

1. Commits as a Communication Tool

A commit is not just a save point — it's a message to every future reader of the codebase (including yourself in six months) about what changed and why. A commit should represent one logical, coherent change; bundling an unrelated bug fix, a refactor, and a new feature into a single commit makes it impossible to revert or review any one of them independently. Commit messages should describe the reasoning ('why') more than the mechanics ('what'), since the diff itself already shows what changed — a message like 'fix stuff' or 'wip' communicates nothing to a future reader trying to understand history.

2. Branching Strategies

Git Flow uses long-lived `develop` and `main` branches plus dedicated `feature`/`release`/`hotfix` branches with strict merge rules — thorough, but heavier process than most modern web teams need. Trunk-based development keeps everyone committing small, frequent changes directly to a single `main` branch (often behind feature flags for anything not ready to ship), minimizing long-lived branches and the painful merge conflicts they tend to accumulate. GitHub Flow sits between the two: short-lived feature branches off `main`, opened as a pull request, reviewed, and merged back quickly — this is the default most small-to-mid-sized teams reach for today.

3. Merging vs. Rebasing

git merge combines two branches' histories with a new merge commit, preserving exactly how history actually happened, including every side branch — accurate, but can leave a tangled, hard-to-read history on an active repo. git rebase replays your branch's commits on top of another branch's current tip, producing a linear history as if you'd started from the latest code — cleaner to read, but it rewrites commit hashes, which is the reason for the cardinal rule: never rebase commits that have already been pushed and might be in use by someone else. A common team convention: rebase your own local feature branch to keep it current with main before opening a PR, but merge (not rebase) when integrating a reviewed PR into main.

BASH

```
# Keep a local feature branch current with main via rebase (before it's shared)
git checkout feature/add-login
git fetch origin
git rebase origin/main

# If conflicts arise, resolve them, then:
git add .
git rebase --continue
```

4. Resolving Merge Conflicts

A conflict occurs when Git can't automatically reconcile changes to the same lines (or nearby lines) in two branches, and it marks the conflicting sections directly in the file with <<<<<<, =====, and >>>>>> markers for you to resolve manually. The reliable process: open each conflicted file, understand what each side was actually trying to do (not just mechanically pick one block), edit the file to the correct final result, remove the conflict markers entirely, then stage the file and continue the merge or rebase. Blindly accepting 'ours' or 'theirs' without reading both sides is how conflicts get 'resolved' into silently broken code.

BASH

```
git status                # see which files have conflicts
# ... manually edit conflicted files, removing <<<<<< ===== >>>>>>
# markers
git add path/to/resolved-file.js
git commit                # completes a merge (or `git rebase --continue` for a
rebase)
```

5. Undoing Mistakes Safely

git revert creates a new commit that undoes a previous commit's changes, preserving history — the safe choice for anything already pushed and shared, since it doesn't rewrite existing commits. git reset moves the branch pointer (optionally discarding commits and/or working-directory changes) and is appropriate for undoing local, unpushed work. git reflog records every place HEAD has pointed, including commits that seem to have vanished after a reset — it's the practical safety net for recovering work that appeared to be lost.

BASH

```
git revert <commit-hash>          # safe: undoes a pushed commit with a new  
git reset --soft HEAD~1          # undo last local commit, keep changes staged  
git reflog                        # find a commit that seems to have disappeared  
git reset --hard <hash-from-reflog> # recover to that exact state
```

6. Pull Request Workflow and Code Review

A pull request bundles a branch's commits for review before they're merged into a shared branch, giving teammates a chance to catch bugs, suggest improvements, and share context before the change ships. Good PR hygiene includes keeping PRs reasonably small and focused (a 2,000-line PR is nearly impossible to review meaningfully), writing a description that explains the why and how to test it, and treating review comments as a collaborative discussion rather than either a rubber stamp or a personal critique.

7. .gitignore and Repository Hygiene

.gitignore tells Git which files/patterns to never track — build output, dependency folders (node_modules), local environment files (.env), and IDE-specific settings are the classic entries, since none of them belong in version control and some (like .env) are an active security risk if committed. If a secret is accidentally committed, simply deleting it in a later commit is not sufficient — it remains fully visible in the repository's history and must be actively purged (or, more practically, the leaked credential must be rotated/invalidated immediately, since history rewriting on a shared repo is itself risky).

Common Pitfalls

- Bundling several unrelated changes into one giant commit, making it impossible to review, revert, or bisect independently.
- Force-pushing or rebasing a branch that other people have already pulled and built work on top of, rewriting history out from under them.
- Committing .env files or other secrets to the repository, requiring credential rotation even after the file is later removed.
- Writing uninformative commit messages ('fix', 'update', 'wip') that give future readers no idea what changed or why.
- Resolving a merge conflict by blindly accepting one side without understanding what both changes were actually trying to do.
- Not pulling the latest changes from the shared branch before starting new work, leading to unnecessarily large and painful merge conflicts later.
- Opening enormous, sprawling pull requests that are effectively unreviewable, leading to rushed or superficial reviews.

Key Takeaways

- A commit should represent one coherent logical change with a message that explains why, not just what.
- Short-lived feature branches merged via pull request (GitHub Flow) is the practical default for most modern teams.
- Never rebase commits that have already been pushed and shared — rebase locally, merge when integrating reviewed work.
- `git revert` is the safe way to undo shared/pushed history; `git reset` is for local, unpushed changes only.
- `git reflog` is the safety net for recovering work that appears lost after a reset or rebase gone wrong.
- A committed secret must be rotated, not just deleted in a later commit — it remains visible in history either way.

Learn More

Git Docs — Branching Basics

<https://git-scm.com/book/en/v2/Git-Branching-Branched-in-a-Nutshell>

Atlassian — Merging vs. Rebasing

<https://www.atlassian.com/git/tutorials/merging-vs-rebasing>

GitHub Docs — About Pull Requests

<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>